

Recorrido del código

`highOrderManufacturedFDAImplicitPETScDistributed`

Generado el 2026-09-17 a partir de
`highOrderManufacturedFDAImplicitPETScDistributed.C` (9931 líneas)

Cómo leer esto. Los listados NO son copias: salen de `\lstinputlisting` apuntando al fuente real, con rangos que este reporte recalcula al regenerarse. El PDF de hoy muestra el código de hoy. Los números de línea del margen son los del archivo.

Índice

1. Qué resuelve este solver	2
2. Andamiaje de PETSc: errores y matriz distribuida	2
3. El offset global de fila	3
4. Nombres de opciones: una sola ortografía, y los de PETSc	4
5. La solución manufacturada	6
6. Bloques de construcción del ensamblado	7
7. Las dos matrices de rigidez	9
8. Resolución del sistema lineal	11
9. Discretización temporal: las dos familias	14
10.El driver de Vm que manejan las EDO	16
11.Reconstrucción a los puntos de integración	18
12.Integración de las EDO, y el paralelismo por hilos	19
13.main(): montaje	21
14.main(): el lazo temporal y los métodos no lineales	22
15.Índice de funciones	23

1. Qué resuelve este solver

Resuelve la ecuación de monodominio con una **solución manufacturada**: se elige de antemano un campo exacto $V_m(x, t)$ y tres estados u_1, u_2, u_3 , se los mete en la ecuación, y lo que sobra se agrega como término fuente. Con eso el error es *medible* y no estimable, que es lo que permite demostrar el orden de precisión — el resultado central del solver.

La ecuación discretizada en el tiempo, con θ eligiendo el esquema ($\theta = 1$ backward Euler, $\theta = 1/2$ Crank–Nicolson), es

$$\left(\frac{M}{\Delta t} - \theta \lambda K\right) V_m^{n+1} = \left(\frac{M}{\Delta t} + (1 - \theta) \lambda K\right) V_m^n + b$$

con M la matriz de masa, K el operador de difusión y b el término fuente más las condiciones de borde. El archivo está organizado casi exactamente en ese orden: primero la infraestructura lineal, después M y K , después la parte no lineal, y al final el lazo temporal.

Dos ejes de paralelismo, y conviene tenerlos separados desde el principio porque el archivo los trata en lugares distintos: **MPI** reparte la malla entre ranks y toca todas las fases, mientras **OpenMP** reparte un lazo de celdas dentro de un rank y sólo toca la integración de las EDO.

2. Andamiaje de PETSc: errores y matriz distribuida

El solver ensambla en Eigen y resuelve en PETSc, así que lo primero del archivo es el puente entre los dos.

`checkPetscError` convierte un código de retorno de PETSc en un `FatalError` de OpenFOAM con el contexto de la llamada. Sin eso un error de PETSc se propaga como un entero que nadie mira.

`buildDistributedMat` es la pieza central del port a MPI: arma la matriz MPIAIJ a partir de los tripletes locales. La matriz es **nLocal** × **nGlobal** — filas propias, columnas globales — que es el layout que PETSc espera. En serie `gRowStart` vale cero y `nLocal == nGlobal`, así que el camino es el mismo.

```
117 void checkPetscError(const PetscErrorCode ierr, const char* context)
118 {
119     if (ierr)
120     {
121         FatalErrorInFunction
122             << "PETSc call failed in " << context
123             << " with error code " << ierr
124             << exit(FatalError);
125     }
126 }
```

Listing 1: Un código de retorno de PETSc pasa a ser un error legible.

```
139 void buildDistributedMat(const SpMat& A, Mat& out)
140 {
141     const PetscInt nLocalRows = static_cast<PetscInt>(A.rows());
142     const PetscInt nGlobalCols = static_cast<PetscInt>(A.cols());
143     const PetscInt rStart = static_cast<PetscInt>(globalRowStart());
144     const PetscInt rEnd = rStart + nLocalRows;
145
146     std::vector<PetscInt> dNnz(nLocalRows, 0);
147     std::vector<PetscInt> oNnz(nLocalRows, 0);
148     for (PetscInt row = 0; row < nLocalRows; ++row)
149     {
150         for (SpMat::InnerIterator it(A, row); it; ++it)
151         {
152             const PetscInt c = static_cast<PetscInt>(it.col());
153             if (c >= rStart && c < rEnd)
154             {
155                 ++dNnz[row];
156             }
157         }
158     }
159 }
```

```

156     }
157     else
158     {
159         ++oNnz[row];
160     }
161 }
162 }
163
164 checkPetscError(MatCreate(PETSC_COMM_WORLD, &out), "MatCreate");
165 checkPetscError
166 (
167     MatSetSizes
168     (
169         out,
170         nLocalRows,
171         nLocalRows,      // local size of the vector this matrix multiplies
172         PETSC_DETERMINE,
173         nGlobalCols
174     ),
175     "MatSetSizes"
176 );
177 checkPetscError(MatSetType(out, MATAIJ), "MatSetType");
178 checkPetscError
179 (
180     MatSeqAIJSetPreallocation
181     (
182         out, 0, dNnz.empty() ? nullptr : dNnz.data()
183     ),
184     "MatSeqAIJSetPreallocation"
185 );
186 checkPetscError
187 (
188     MatMPIAIJSetPreallocation
189     (
190         out,
191         0, dNnz.empty() ? nullptr : dNnz.data(),
192         0, oNnz.empty() ? nullptr : oNnz.data()
193     ),
194     "MatMPIAIJSetPreallocation"
195 );
196 checkPetscError
197 (
198     MatSetOption(out, MAT_NEW_NONZERO_ALLOCATION_ERR, PETSC_FALSE),

```

Listing 2: Tripletas locales a matriz distribuida. Recortado. (*recortado; sigue en el fuente.*)

3. El offset global de fila

`gRowStart` es el desplazamiento de la primera fila de este rank, o sea `globalIndex::localStart()`. Es constante para toda la corrida — una malla, una descomposición — así que se fija una vez en `main()` en vez de arrastrarlo por la firma de cada rutina.

Vale la pena detenerse acá porque es donde vivían cuatro bugs del port, todos del mismo tipo: escribir una columna *local* donde iba una *global*. En serie `gRowStart` es cero y `toGlobal()` es la identidad, así que **ninguno de los cuatro era visible sin correr en paralelo**. El test **M09** compara K entrada por entrada contra el ensamblado serial deshaciendo la permutación con `cellProcAddressing`, y es lo único que puede verlos: un fingerprint de nnz y sumas de fila es invariante a mover una entrada de columna.

Justamente por eso el valor no se lee directo sino a través de `globalRowStart()`, que **se niega a servir el cero inicial**. La asignación ocurre en `main()`, después de construir el interpolador, y todas las inserciones de fila en PETSc la leen. Un camino de código que llegara a una matriz antes de ese punto usaría offset cero *en todos* los ranks, o sea que los ranks > 0 escribirían sus filas encima de las del rank 0: operador corrompido y respuesta plausible pero equivocada, nunca un error. Hoy nada lo hace; el accesor lo vuelve imposible en vez de meramente improbable.

```

80 // Added for cardiacFoam: global row offset of this rank's block of cells,
81 // i.e. globalIndex::localStart(). Constant for the whole run (one mesh, one
82 // decomposition), so it is set once in main() rather than threaded through
83 // every linear-solver signature. Zero in serial.
84 label gRowStart = 0;
85 bool gRowStartSet = false;
86
87 // Modified for cardiacFoam: read through an accessor that refuses to serve
88 // the initial 0. The assignment happens in main, after the interpolator is
89 // built, and every PETSc row insertion reads it. A future code path that
90 // touched a matrix before that point would offset by 0 on EVERY rank, so
91 // ranks > 0 would write their rows on top of rank 0's: a corrupt operator
92 // and a plausible wrong answer, never an error. Nothing does that today -
93 // this makes it impossible rather than merely unlikely.
94 label globalRowStart()
95 {
96     if (!gRowStartSet)
97     {
98         FatalErrorInFunction
99         << "The global row offset was read before it was set."
100         << " It is assigned once in main from"
101         << " LREinterp_Vm.globalCells().localStart(); reaching a PETSc"
102         << " matrix before that offsets every rank's rows by 0 and"
103         << " corrupts the operator."
104         << exit(FatalError);
105     }
106
107     return gRowStart;
108 }

```

Listing 3: El offset, y el accesor que se niega a servirlo sin asignar.

4. Nombres de opciones: una sola ortografía, y los de PETSc

Cada opción del diccionario tiene exactamente una ortografía, comparada distinguiendo mayúsculas, y cualquier otra cosa detiene la corrida al arrancar con la lista de valores válidos. Lo hace `requireOneOf`, que se llama en `createFields.H` justo después de leer cada clave.

Reemplazó a una capa que pasaba todo a minúsculas y acumulaba alias (`sparselu/lu`, `picard`, `localDiagonal`, `RKT45`, `consistentH0`, `jakobi`...). No era cosmético: así se escondió una respuesta equivocada. Con `linearSolver SparseLU`, el camino de `diagonalIion` forzaba un solve directo, pero el solver cacheado de Picard recibía `preonly` con el PC por defecto, `ilu`: una aplicación de ILU haciéndose pasar por la inversa. Picard convergía al punto fijo del operador equivocado — V_m con un error 4,6 veces mayor, cero pasos no convergidos, ninguna advertencia. `resolvePetscKspPc` resuelve ahora ese par en un solo lugar para los tres caminos PETSc.

Quedan dos vocabularios separados. `linearSolver` es neutral (`GMRES`, `BiCGSTAB`, `SparseLU`); las claves de PETSc usan los nombres propios de PETSc de listas cerradas. `petscKspTypeForLinearSolver` es lo único que traduce de uno a otro, para el valor por defecto de `petscLinearKspType`.

La función de esta sección que importa para interpretar resultados es `petscParallelPcTypeName`.

`ilu` y `lu` no tienen implementación `MPIAIJ`, así que en paralelo PETSc los envuelve en **block-Jacobi**, y el bloque *es* la partición. Es decir que a `np=1` y a `np=2` no se resuelve con el mismo preconditionador: son dos algoritmos distintos. `hypre` y `gamg` sí son paralelos nativos, pero su *coarsening* también depende de cómo quedó cortada la malla.

Sólo `jakobi` es el mismo operador a cualquier número de ranks, y ésa es la razón de que la suite lo fije en todos los tests de invariancia. El test **M07** lo cuantifica: la desviación entre `np=1` y `np=2` es **1.26e-11** con `jakobi` contra **3.60e-05** con `ilu`, un factor de 2.86 millones.

Para producción la respuesta es otra y está en **M18**: ahí lo que importa es el tiempo, y el ranking cambia con el paso de tiempo.

```

343 void requireOneOf
344 (
345     const char* key,
346     const word& value,
347     std::initializer_list<const char*> valid
348 )
349 {
350     for (const char* v : valid)
351     {
352         if (value == v)
353         {
354             return;
355         }
356     }
357
358     std::string options;
359     for (const char* v : valid)
360     {
361         if (!options.empty())
362         {
363             options += ", ";
364         }
365         options += v;
366     }
367
368     FatalErrorInFunction
369     << "Unknown " << key << ": " << value << nl
370     << "Valid options are " << options.c_str()
371     << " (exact spelling, case-sensitive)."
372     << exit(FatalError);
373 }

```

Listing 4: Una sola ortografía por opción, validada al arrancar.

```

409 void resolvePetscKspPc
410 (
411     const word& linearSolver,
412     const word& petscKspType,
413     const word& petscPcType,
414     word& kspType,
415     word& pcType
416 )
417 {
418     kspType = petscKspType;
419     pcType = petscPcType;
420
421     if (linearSolver == "SparseLU")
422     {
423         kspType = "preonly";
424         pcType = "lu";
425     }
426 }

```

Listing 5: El par (KSP, PC) de un linearSolver, en un solo lugar.

```

460 std::string petscParallelPcTypeName(const word& pcType, std::string& subPc)
461 {
462     const std::string s = std::string(pcType);
463     subPc.clear();
464
465     if (Pstream::parRun() && (s == "ilu" || s == "lu"))
466     {
467         subPc = s;
468         return "bjacobi";
469     }
470
471     return s;
472 }

```

Listing 6: La sustitución que hace que ilu deje de ser ilu en paralelo.

5. La solución manufacturada

Acá está lo que hace que este solver pueda demostrar un orden en vez de estimarlo. `exactVm` y las tres `exactU` dan el campo elegido; `computeF` y `computeG` son sus factores espaciales, escritos por dimensión.

`vmSourcePDE` es el término que se agrega a la ecuación para que ese campo sea solución exacta: se obtiene metiendo la solución elegida en el operador y despejando lo que falta.

`computeBeta` lee `conductivity[0]`, o sea la celda cero, y eso sólo es válido con conductividad **uniforme**: en una malla descompuesta la celda cero es una celda física distinta en cada rank. No es un item abierto sino un **supuesto asumido** — este solver existe para este caso manufacturado, donde D es uniforme por diseño. Queda registrado acá porque es lo primero que habría que tocar si alguna vez entraran fibras, que hacen D no uniforme por construcción.

Hay dos soluciones manufacturadas, y la clave `manufacturedSolution` elige. La de siempre, `sines`, es $\sqrt{1+t} \cos(\pi x) \cos(2\pi y) \dots$: suave y no polinómica, así que ninguna reconstrucción de acá es exacta sobre ella y el orden *espacial* medido es el del esquema.

La otra, `uniform`, tiene todos los factores espaciales iguales a uno. Existe por una razón puntual: toda reconstrucción, cuadratura y flujo de este solver es exacto sobre un campo constante, así que el error espacial es cero a nivel de redondeo y el error contra la solución exacta es **puramente temporal**. Es la única forma de ajustar un orden temporal contra la solución analítica en vez de contra otra solución numérica, y es lo que mide **M22**. A malla fija con `sines` eso es imposible: el error espacial ($2,4 \times 10^{-5}$ en hexa $N = 40$) entierra al temporal, que para BDF4 es $\sim 10^{-10}$.

Lo que cuesta, dicho para que nadie lea de más el resultado: el laplaciano de una constante es cero, así que bajo `uniform` la difusión no participa y lo que se verifica es la integración temporal del sistema de reacción. La difusión la cubre **M02** por autoconvergencia. Y `computeBeta` devuelve cero en ese modo, porque no hay laplaciano que cancelar.

```
1456 scalar exactVm(const point& p, const scalar t, const label dim)
1457 {
1458     return std::sqrt(1.0 + t)*computeF(p, dim);
1459 }
```

Listing 7: El campo exacto.

```
1403 scalar computeF(const point& p, const label dim)
1404 {
1405     // Exactly representable by every reconstruction here, so the spatial
1406     // error vanishes and what is left is the temporal error.
1407     if (mmsProfile == mmsUniform)
1408     {
1409         return 1.0;
1410     }
1411
1412     const scalar pi = constant::mathematical::pi;
1413
1414     if (dim == 1)
1415     {
1416         return std::cos(pi*p.x());
1417     }
1418     else if (dim == 2)
1419     {
1420         return std::cos(pi*p.x())*std::cos(2.0*pi*p.y());
1421     }
1422
1423     return
1424         std::cos(pi*p.x())
1425         *std::cos(2.0*pi*p.y())
1426         *std::cos(3.0*pi*p.z());
1427 }
```

Listing 8: El factor espacial, y dónde el modo uniform lo vuelve 1.

```

1550     scalar vmSourcePDE
1551     (
1552         const scalar V,
1553         const scalar u1,
1554         const scalar u2,
1555         const scalar u3,
1556         const scalar beta,
1557         const scalar chiVal,
1558         const scalar CmVal
1559     )
1560     {
1561         return -ionicCurrentPDE(V, u1, u2, u3, beta, chiVal, CmVal);
1562     }

```

Listing 9: El término que lo convierte en solución.

```

1568     scalar computeBeta(const volTensorField& conductivity, const label dim)
1569     {
1570         // A constant has zero Laplacian, so there is nothing for beta to
1571         // cancel and the ionic current reduces to the term that supplies
1572         // dV/dt.
1573         if (mmsProfile == mmsUniform)
1574         {
1575             return 0.0;
1576         }
1577
1578         const tensor& D = conductivity[0];
1579         const scalar pi2 = sqr(constant::mathematical::pi);
1580
1581         if (dim == 1)
1582         {
1583             return -pi2*D.xx();
1584         }
1585         else if (dim == 2)
1586         {
1587             return -pi2*(D.xx() + 4.0*D.yy());
1588         }
1589
1590         return -pi2*(D.xx() + 4.0*D.yy() + 9.0*D.zz());
1591     }

```

Listing 10: Lee la celda 0: sólo vale con D uniforme.

6. Bloques de construcción del ensamblado

Antes de las dos matrices de rigidez vienen las piezas que las dos comparten.

`addTripletIfNeeded` es el punto por el que pasa *toda* entrada de la matriz. Descarta las que valen menos que `SMALL`, lo que hace que el patrón de esparsidad dependa del valor y, por lo tanto, de la partición al nivel del redondeo.

`addCellGradientDotCoeffs` agrega la fila de gradiente de una celda proyectada sobre un vector. Usa `globalCellStencils()`, y ahí está el problema abierto más grande del proyecto: sobre las celdas pegadas a un corte de procesador ese stencil **pierde un miembro** (17 en serie contra 16 en paralelo). Como el ajuste MLS se rehace sobre el conjunto reducido, no cambia sólo el término que falta — **refita toda la fila**. Eso es lo que hace fallar a **M11** y, río abajo, a **M15**.

`exchangeCoupledFaceRows` es la que permite que el término de estabilización cruce un corte: intercambia con el rank vecino las filas que hacen falta del otro lado. Es **colectiva**, y está deliberadamente fuera del lazo de chunks — adentro se trabajaría en cuanto dos ranks tuvieran una cantidad distinta de chunks.

```

2301     void addTripletIfNeeded
2302     (
2303         std::vector<Triplet>& triplets,
2304         const label row,
2305         const label col,

```

```

2306     const scalar value
2307 )
2308 {
2309     if (mag(value) > SMALL)
2310     {
2311         triplets.emplace_back(row, col, value);
2312     }
2313 }

```

Listing 11: Por acá pasa toda entrada de la matriz.

```

2621 void addCellGradientDotCoeffs
2622 (
2623     std::vector<Triplet>& triplets,
2624     const label row,
2625     const scalar scale,
2626     const label cellI,
2627     const vector& d,
2628     const highOrderInterp& LREInterp
2629 )
2630 {
2631     if (mag(scale) <= SMALL)
2632     {
2633         return;
2634     }
2635
2636     // Modified for cardiacFoam: matrix columns are global cell indices.
2637     const globalIndex& gc = LREInterp.globalCells();
2638
2639     const CompactListList<label>& stencils = LREInterp.globalCellStencils();
2640     const CompactListList<vector>& gradCoeffs = LREInterp.QRGradCoeffs();
2641     // Modified for cardiacFoam: CompactListList::operator[] returns a
2642     // UList VIEW by value, not a List. Binding it to a const labelList&
2643     // compiles but yields an EMPTY list, silently dropping the whole
2644     // stencil. Bind to the returned view type instead.
2645     const UList<label> curStencil = stencils[cellI];
2646     const label selfCoeffI = curStencil.size();
2647
2648     forAll(curStencil, cI)
2649     {
2650         addTripletIfNeeded
2651         (
2652             triplets,
2653             row,
2654             curStencil[cI],
2655             scale*(gradCoeffs[cellI][cI] & d)
2656         );
2657     }
2658
2659     addTripletIfNeeded
2660     (
2661         triplets,
2662         row,
2663         gc.toGlobal(cellI),
2664         scale*(gradCoeffs[cellI][selfCoeffI] & d)
2665     );
2666 }

```

Listing 12: La fila de gradiente. Recortada.

```

2822 void exchangeCoupledFaceRows
2823 (
2824     const fvMesh& mesh,
2825     const std::function
2826     <
2827         void(const label patchI, const label faceI, labelList&, scalarField&)
2828     >& buildRow,
2829     List<labelList>& nbrCols,
2830     List<scalarField>& nbrCoeffs
2831 )
2832 {
2833     nbrCols.setSize(mesh.nBoundaryFaces());
2834     nbrCoeffs.setSize(mesh.nBoundaryFaces());
2835
2836     if (!Pstream::parRun())

```

```

2837 {
2838     return;
2839 }
2840
2841 PstreamBuffers pBufs(Pstream::commsTypes::nonBlocking);
2842
2843 forAll(mesh.boundary(), patchI)
2844 {
2845     const fvPatch& patch = mesh.boundary()[patchI];
2846     const processorFvPatch* ppPtr = isA<processorFvPatch>(patch);
2847
2848     if (!ppPtr)
2849     {
2850         continue;
2851     }
2852
2853     List<labelList> sendCols(patch.size());
2854     List<scalarField> sendCoeffs(patch.size());
2855
2856     forAll(patch, faceI)
2857     {
2858         buildRow(patchI, faceI, sendCols[faceI], sendCoeffs[faceI]);
2859     }
2860
2861     UOPstream toNbr(ppPtr->neighbProcNo(), pBufs);
2862     toNbr << sendCols << sendCoeffs;
2863 }
2864
2865 pBufs.finishedSends();
2866
2867 forAll(mesh.boundary(), patchI)
2868 {
2869     const fvPatch& patch = mesh.boundary()[patchI];
2870     const processorFvPatch* ppPtr = isA<processorFvPatch>(patch);
2871
2872     if (!ppPtr)
2873     {
2874         continue;
2875     }
2876
2877     List<labelList> recvCols;
2878     List<scalarField> recvCoeffs;
2879
2880     UIPstream fromNbr(ppPtr->neighbProcNo(), pBufs);
2881     fromNbr >> recvCols >> recvCoeffs;

```

Listing 13: Intercambio de filas a través de un corte. Recortada. (*recortado; sigue en el fuente.*)

7. Las dos matrices de rigidez

El solver tiene dos ensamblados del operador de difusión y la tríada del caso elige cuál corre.

`assembleStandardOrthogonalStiffnessMatrix` es el de bajo orden: flujo de dos puntos entre celdas vecinas. Usa el **promedio** $\frac{1}{2}(D_{own} + D_{nei})$ como tensor de cara.

`assembleHighOrderStiffnessMatrix` es el de alto orden, sobre el stencil LRE con cuadratura en las caras. Acá estuvo el defecto más grande que encontró la verificación: usaba `conductivity[own]` **en las dos filas** de una cara interna, en vez del promedio. Con D uniforme los dos tensores coinciden y se cancela idénticamente, así que fue invisible durante toda la vida del código; con D no uniforme hacía que K **cambiara un 21 % al descomponer la malla**, porque al cortar, cada rank es dueño de su lado de la cara. Está corregido, y **M15** lo mide: de 2.108e-01 a 4.736e-15.

Las dos ensamblan **por chunks** de caras, y la clave del diccionario `assemblyFaceChunk` controla el tamaño. Es control de capacidad solamente: el resultado no debe depender de él, y **M12** lo verifica — con tolerancia y no bit a bit, porque cambiar el tamaño de bloque cambia el orden de suma en punto flotante.

```

3083 void assembleStandardOrthogonalStiffnessMatrix

```

```

3084 (
3085     const fvMesh& mesh,
3086     const volTensorField& conductivity,
3087     const highOrderInterp& LREInterp,
3088     const scalar stabilisationAlpha,
3089     SpMat& K
3090 )
3091 {
3092     const vectorField& C = mesh.C();
3093     const scalarField& V = mesh.V();
3094     const labelUList& owner = mesh.owner();
3095     const labelUList& neighbour = mesh.neighbour();
3096
3097     // Modified for cardiacFoam: local rows, GLOBAL columns, as in
3098     // assembleHighOrderStiffnessMatrix. This assembly still used local
3099     // columns for its orthogonal terms - and, through
3100     // addCellGradientDotCoeffs, global ones for its stabilisation terms -
3101     // which agree only in serial.
3102     const globalIndex& gc = LREInterp.globalCells();
3103
3104     std::vector<Triplet> triplets;
3105     triplets.reserve(4*mesh.nFaces());
3106
3107     forAll(neighbour, faceI)
3108     {
3109         const label own = owner[faceI];
3110         const label nei = neighbour[faceI];
3111
3112         const tensor Df = 0.5*(conductivity[own] + conductivity[nei]);
3113         const scalar a = orthogonalDiffusionCoeff(mesh.Sf()[faceI], C[nei] - C[own], Df);
3114
3115         addTripletIfNeeded(triplets, own, gc.toGlobal(own), -a/max(V[own], SMALL));
3116         addTripletIfNeeded(triplets, own, gc.toGlobal(nei), a/max(V[own], SMALL));
3117         addTripletIfNeeded(triplets, nei, gc.toGlobal(own), a/max(V[nei], SMALL));
3118         addTripletIfNeeded(triplets, nei, gc.toGlobal(nei), -a/max(V[nei], SMALL));
3119     }
3120
3121     const surfaceVectorField& Cf = mesh.Cf();
3122
3123     if (stabilisationAlpha > SMALL)
3124     {
3125         forAll(neighbour, faceI)
3126         {
3127             const label own = owner[faceI];
3128             const label nei = neighbour[faceI];
3129
3130             const vector Sf = mesh.Sf()[faceI];
3131             const scalar area = mag(Sf) + VSMALL;
3132             const vector n = Sf/area;
3133             const vector dPN = C[nei] - C[own];
3134             const tensor Df = 0.5*(conductivity[own] + conductivity[nei]);
3135             const scalar a =
3136                 stabilisationFaceCoeff
3137                 (
3138                     stabilisationAlpha,
3139                     area,
3140                     Df,
3141                     n,
3142                     dPN

```

Listing 14: Bajo orden: promedio en la cara. Recortado. (recortado; sigue en el fuente.)

```

3392 void assembleHighOrderStiffnessMatrix
3393 (
3394     const fvMesh& mesh,
3395     const volTensorField& conductivity,
3396     const highOrderInterp& LREInterp,
3397     const scalar stabilisationAlpha,
3398     const label tripletsPerFaceReserve,
3399     const label faceChunk,
3400     SpMat& K
3401 )
3402 {
3403     // Chunked assembly: avoids accumulating ~434M triplets in a single
3404     // std::vector (which would peak at ~19 GB transient due to repeated

```

```

3405 // capacity-doubling reallocations on large 3D unstructured tet
3406 // meshes). The chunked variant bounds the triplet buffer to a fixed
3407 // size per chunk and folds each chunk into the global K via sparse
3408 // matrix addition. Numerically equivalent to the monolithic version:
3409 //  $K = \sum_i K\_chunk\_i$ , and addTripletIfNeeded filters by individual
3410 // value magnitude (not by accumulated value), so the chunking is
3411 // transparent to the final result.
3412 const bool twoD = mesh.nGeometricD() == 2;
3413 const vectorField& C = mesh.C();
3414 const surfaceVectorField& Cf = mesh.Cf();
3415 const CompactListList<point>& faceQP = LREInterp.faceQuadPoints();
3416 const CompactListList<scalar>& faceQW = LREInterp.faceQuadWeightPhysical();
3417 const CompactListList<label>& faceStencils = LREInterp.globalFaceStencils();
3418 const List<CompactListList<vector>>& faceGradCoeffs =
3419     LREInterp.QRGradFaceGPCoeffs();
3420
3421 const scalarField& V = mesh.V();
3422 const labelUList& owner = mesh.owner();
3423 const labelUList& neighbour = mesh.neighbour();
3424
3425 const label nCells = mesh.nCells();
3426 // Modified for cardiacFoam: local rows, global columns (MPIAIJ layout).
3427 const globalIndex& gc = LREInterp.globalCells();
3428 const label nGlobalCells = gc.totalSize();
3429 const label nInternalFaces = neighbour.size();
3430
3431 std::vector<Triplet> triplets;
3432 // Reserve for a full chunk without forcing the tet/stabilised worst
3433 // case on flux-only hex runs. The cap is selected by the caller from
3434 // memoryOptimization controls; it changes capacity only, not assembly.
3435 triplets.reserve(faceChunk * tripletsPerFaceReserve);
3436
3437 SpMat Klocal(nCells, nGlobalCells);
3438 bool kInitialised = false;
3439
3440 auto flushChunk =
3441 [&]()
3442 {
3443     if (triplets.empty())
3444     {
3445         return;
3446     }
3447     Klocal.setZero();
3448     Klocal.setFromTriplets(triplets.begin(), triplets.end());
3449     Klocal.makeCompressed();
3450
3451     if (!kInitialised)

```

Listing 15: Alto orden, sobre el stencil LRE. Recortado. (recortado; sigue en el fuente.)

8. Resolución del sistema lineal

`solveSparseSystem` despacha entre PETSc y Eigen según `linearSolverBackend`. En paralelo sólo el camino de PETSc sirve, y ahora eso **se hace cumplir** en vez de sólo estar comentado: la matriz es $n_{\text{Local}} \times n_{\text{Global}}$, o sea rectangular, y las factorizaciones de Eigen necesitan una matriz cuadrada. Sin el guard, `SparseLU` moría en un `eigen_assert` que nombra un archivo interno de la librería — ni este solver, ni la clave del diccionario que lo eligió, ni la palabra “paralelo”. Medido a $np=2$: el error ahora dice 200 local rows and 400 global columns.

Hay además un GMRES escrito a mano (`solveGMRES`) que queda como camino de respaldo cuando no se usa PETSc. Ése tiene su propio guard, y por un motivo **distinto**: no hay matriz que pueda ser rectangular, pero sus productos internos ($V[i].\text{dot}(w)$, $w.\text{norm}()$) los calcula Eigen sobre el vector *local* y nunca se reducen. Cada rank armaría su propia base de Krylov y su propio test de convergencia, mientras el producto matriz-vector que llama *sí* es colectivo (`MatMult` de PETSc). Dos ranks que salgan del lazo en distinta iteración se traban ahí: no es un abort, es un cuelgue.

Un tercer punto de la misma familia: `usesPetscBackend` **valida** el nombre en vez de tratarlo como

una lista blanca de dos entradas cuyo `else` era Eigen. Cualquier valor que no fuera `petsc/ksp` — un tipo en un diccionario, por ejemplo — mudaba la corrida entera al backend serial sin una línea en el log. Es la misma falla que `nonlinearMethod Picrad` en el solver electro, que colapsaba una matriz de métodos sobre un solo método.

Qué solver de Krylov corresponde lo decide la simetría del operador, y está medida en **M10**: el de bajo orden es simétrico a 8.4e-16, el de alto orden **no** — 7.9 % de asimetría relativa, con 300 de 4292 entradas sin transpuesta. Con la tríada de alto orden hay que usar `gmres`. `cg` no falla siempre ahí, que es lo que lo hace peligroso: sobrevive mientras le alcancen pocas iteraciones y se cae cuando necesita más (**E13** lo muestra fallando con `jacobi` y `gamg`).

```

385 bool usesPetscBackend(const word& backend)
386 {
387     requireOneOf("linear solver backend", backend, {"PETSc", "Eigen"});
388     return backend == "PETSc";
389 }
```

Listing 16: El nombre del backend, validado.

```

4149 EigVec solveSparseSystem
4150 (
4151     const SpMat& A,
4152     const EigVec& b,
4153     const word& linearSolverBackend,
4154     const word& linearSolver,
4155     const word& petscKspType,
4156     const word& petscPcType,
4157     const label petscRestart,
4158     const word& petscOptionsPrefix,
4159     const bool petscUseOptions,
4160     const scalar tol,
4161     const label maxIter,
4162     label& linearIterations,
4163     scalar& linearError
4164 )
4165 {
4166     if (usesPetscBackend(linearSolverBackend))
4167     {
4168         word kspType;
4169         word pcType;
4170         resolvePetscKspPc
4171         (
4172             linearSolver, petscKspType, petscPcType, kspType, pcType
4173         );
4174
4175         PetscKspMatrixSolver solver;
4176         solver.reset
4177         (
4178             A,
4179             kspType,
4180             pcType,
4181             tol,
4182             maxIter,
4183             petscRestart,
4184             petscOptionsPrefix,
4185             petscUseOptions
4186         );
4187
4188         return solver.solve(b, linearIterations, linearError);
4189     }
4190
4191     // Modified for cardiacFoam: the Eigen backend is serial only, and the
4192     // comment above this function said so without enforcing it. K carries
4193     // LOCAL rows and GLOBAL columns (the MPIAIJ layout, see
4194     // assembleStiffnessMatrix), so in parallel it is rectangular by design
4195     // and Eigen's SparseLU/BiCGSTAB die on an eigen_assert naming a file
4196     // inside the library - not this solver, not the key that selected them,
4197     // and not the word "parallel".
4198     if (Pstream::parRun())
4199     {
4200         FatalErrorInFunction
4201             << "linearSolverBackend = " << linearSolverBackend
4202             << " selects the Eigen backend, which is serial only." << nl

```

```

4203         << "The assembled matrix has " << A.rows()
4204         << " local rows and " << A.cols()
4205         << " global columns, so Eigen would be handed a rectangular"
4206         << " system." << nl
4207         << "Set 'linearSolverBackend PETSc;' to run on "
4208         << Pstream::nProcs() << " ranks."

```

Listing 17: El despacho entre backends, con el guard de paralelo. Recortado. (*recortado; sigue en el fuente.*)

```

4070 EigVec solveSparseSystemEigen
4071 (
4072     const SpMat& A,
4073     const EigVec& b,
4074     const word& linearSolver,
4075     const scalar tol,
4076     const label maxIter,
4077     label& linearIterations,
4078     scalar& linearError
4079 )
4080 {
4081     if (linearSolver == "SparseLU")
4082     {
4083         Eigen::SparseLU<SpMat> solver;
4084         solver.analyzePattern(A);
4085         solver.factorize(A);
4086
4087         if (solver.info() != Eigen::Success)
4088         {
4089             FatalErrorInFunction
4090                 << "SparseLU factorization failed"
4091                 << exit(FatalError);
4092         }
4093
4094         EigVec x = solver.solve(b);
4095         linearIterations = 1;
4096         linearError = 0.0;
4097
4098         if (solver.info() != Eigen::Success)
4099         {
4100             FatalErrorInFunction
4101                 << "SparseLU solve failed"
4102                 << exit(FatalError);
4103         }
4104
4105         return x;
4106     }
4107     else if (linearSolver == "BiCGSTAB")
4108     {
4109         Eigen::BiCGSTAB<SpMat, Eigen::IncompleteLUT<scalar>> solver;
4110         solver.setTolerance(tol);
4111         solver.setMaxIterations(maxIter);
4112         solver.compute(A);
4113
4114         if (solver.info() != Eigen::Success)
4115         {
4116             FatalErrorInFunction
4117                 << "BiCGSTAB setup failed"
4118                 << exit(FatalError);
4119         }
4120
4121         EigVec x = solver.solve(b);
4122         linearIterations = solver.iterations();
4123         linearError = solver.error();
4124
4125         if (solver.info() != Eigen::Success)
4126         {
4127             FatalErrorInFunction
4128                 << "BiCGSTAB solve failed"
4129                 << exit(FatalError);

```

Listing 18: El camino serial. Recortado. (*recortado; sigue en el fuente.*)

9. Discretización temporal: las dos familias

El solver ofrece dos familias de integrador temporal, y `implicitScheme` elige entre ellas. Con el orden que cada una debe entregar:

<code>implicitScheme</code>	orden	estabilidad
<code>backwardEuler</code>	$O(\Delta t)$	A y L-estable
<code>crankNicolson</code>	$O(\Delta t^2)$	A-estable, no L-estable
<code>BDF1</code>	idéntico a <code>backwardEuler</code> , bit a bit	A y L
<code>BDF2</code>	$O(\Delta t^2)$	A y L-estable
<code>BDF3</code>	$O(\Delta t^3)$	$A(86,03^\circ)$
<code>BDF4</code>	$O(\Delta t^4)$	$A(73,35^\circ)$

`timeSchemeCoeffs` traduce el nombre a los coeficientes. Lo que hay que ver en esa función es que **las dos familias terminan en el mismo par de operadores**: $A = (a_0/\Delta t)M - \theta L$ y $B = M/\Delta t + (1 - \theta)L$. BDF cambia *sólo* el escalar que multiplica a M , así que el ensamblado, el patrón de esparsidad y la factorización cacheada quedan intactos. Y el historial entra por un *único* producto por la matriz de masa, porque la combinación $H^n = -\sum_{j \geq 1} a_j V^{n+1-j}$ se arma como campo antes de aplicar B : un paso BDF- k cuesta un producto matriz-vector, no k .

BDF5 y BDF6 no se ofrecen. Pasada la segunda barrera de Dahlquist ningún multipaso es A-estable, y la cuña se cierra de 86° y 73° en BDF3/4 a $51,8^\circ$ y $17,8^\circ$ en BDF5/6.

El arranque es la parte que engaña. Un método de k pasos necesita valores iniciales precisos a $O(\Delta t^k)$ respecto del sistema *semi-discreto*, no de la PDE. Sembrar con la solución manufacturada da $O(\tau_h \Delta t)$ — primer orden — porque la solución exacta de la PDE no satisface las ecuaciones semi-discretas. Nada avisa: la corrida termina bien y sólo el orden sale mal, que es por lo que `bdfStartup` tiene hoy **ESDIRK3 por defecto**.

Con ese arranque los órdenes son los nominales. Medido a $N = 40$ por las dos vías que usa la suite — **M02**, autoconvergencia de Cauchy sobre la solución de senos con la difusión activa, y **M22**, error contra la solución analítica sobre la MMS uniforme:

<code>implicitScheme</code>	nominal	Cauchy (M02)	vs analítica (M22)
<code>backwardEuler</code> y <code>BDF1</code>	1	1,000	1,000
<code>crankNicolson</code>	2	1,994	2,000
<code>BDF2</code>	2	1,979	1,992
<code>BDF3</code>	3	2,932	2,971
<code>BDF4</code>	4	3,890	3,941

`exact` y `constant` se conservan, pero **sólo** para que el costo de un mal arranque siga siendo medible: es lo que hace **M20**, aislando el efecto como $e(\Delta t) = \|V_{\text{arranque}} - V_{\text{ESDIRK3}}\|$ al mismo Δt y la misma malla, donde el error propio de BDF3 se cancela. Los dos modos resultan $O(\Delta t)$, y difieren donde la teoría dice: el de `constant` no cambia al refinar la malla (razón 1,00) y el de `exact` cae 15,5 veces por refinamiento, porque su coeficiente es τ_h .

Por eso existe `esdirk3Tableau`, y **sólo** por eso: ESDIRK no es seleccionable como esquema. Es autoarrancable, que es lo que el arranque pide, pero tiene orden de etapa 2 y sufre reducción de orden — ESDIRK4 midió 3,03 donde BDF4 llega a 3,89; esa cifra de ESDIRK4 es anterior al arreglo del lazo temporal y no se volvió a medir — y cada etapa reintegra las EDO, que es el 74–99 % del tiempo en el solver fisiológico.

```
1805 | TimeSchemeCoeffs timeSchemeCoeffs(const word& scheme)
1806 | {
```

```

1807 TimeSchemeCoeffs c;
1808
1809 if (scheme == "backwardEuler")
1810 {
1811     c.theta = 1.0;
1812     c.bdfOrder = 0;
1813     return c;
1814 }
1815
1816 if (scheme == "crankNicolson")
1817 {
1818     c.theta = 0.5;
1819     c.bdfOrder = 0;
1820     return c;
1821 }
1822
1823 // BDF is fully implicit in K, so theta = 1, and the source is wanted at
1824 // t^{n+1} only - one source evaluation per step FEWER than
1825 // Crank-Nicolson needs.
1826 c.theta = 1.0;
1827
1828 if (scheme == "BDF1")
1829 {
1830     c.bdfOrder = 1;
1831     c.a[0] = 1.0;
1832     c.a[1] = -1.0;
1833 }
1834 else if (scheme == "BDF2")
1835 {
1836     c.bdfOrder = 2;
1837     c.a[0] = 3.0/2.0;
1838     c.a[1] = -2.0;
1839     c.a[2] = 1.0/2.0;
1840 }
1841 else if (scheme == "BDF3")
1842 {
1843     c.bdfOrder = 3;
1844     c.a[0] = 11.0/6.0;
1845     c.a[1] = -3.0;
1846     c.a[2] = 3.0/2.0;
1847     c.a[3] = -1.0/3.0;
1848 }
1849 else if (scheme == "BDF4")
1850 {
1851     c.bdfOrder = 4;
1852     c.a[0] = 25.0/12.0;
1853     c.a[1] = -4.0;
1854     c.a[2] = 3.0;
1855     c.a[3] = -4.0/3.0;
1856     c.a[4] = 1.0/4.0;
1857 }
1858 else
1859 {
1860     FatalErrorInFunction
1861     << "Unknown implicitScheme: " << scheme << nl
1862     << "Valid options are backwardEuler, crankNicolson, BDF1,"
1863     << " BDF2, BDF3 and BDF4." << nl
1864     << "ESDIRK is not a scheme here: it exists only to supply the"

```

Listing 19: Los coeficientes de las dos familias. Recortado. *(recortado; sigue en el fuente.)*

```

1765 EsdirkTableau esdirk3Tableau()
1766 {
1767     EsdirkTableau tb;
1768     tb.nStages = 4;
1769     tb.order = 3;
1770     tb.gamma = 0.435866521508458999416;
1771
1772     forAll(tb.c, i)
1773     {
1774         tb.c[i] = 0.0;
1775         forAll(tb.a[i], j)
1776         {
1777             tb.a[i][j] = 0.0;
1778         }
1779     }

```

```

1779     }
1780
1781     const scalar g = tb.gamma;
1782
1783     tb.c[0] = 0.0;
1784     tb.c[1] = 2.0*g;
1785     tb.c[2] = 0.6;
1786     tb.c[3] = 1.0;
1787
1788     tb.a[1][0] = g;
1789     tb.a[1][1] = g;
1790
1791     tb.a[2][0] = 0.257648246066427245800;
1792     tb.a[2][1] = -0.093514767574886245216;
1793     tb.a[2][2] = g;
1794
1795     tb.a[3][0] = 0.187641024346723825161;
1796     tb.a[3][1] = -0.595297473576954948048;
1797     tb.a[3][2] = 0.971789927721772123471;
1798     tb.a[3][3] = g;
1799
1800     return tb;
1801 }

```

Listing 20: La tabla del arranque, derivada de las condiciones de orden.

```

8289 // Added for cardiacFoam: the ESDIRK startup step. This runs ONLY for
8290 // the first bdfOrder-1 steps of a BDF run, to build the history; it is
8291 // not reachable as a scheme in its own right (see the note on
8292 // needEsdirkOperator above for why not).
8293 //
8294 // Stage 1 is explicit, Y_1 = V^n. For i >= 2 the stage solves
8295 //
8296 //   A_g Y_i = (1/g) [ B V^n + sum_{j<i} a_{ij} F_j ] + bc(t_i) + s(Y_i)
8297 //
8298 // with A_g = M/(g dt) - lapScale K, B = M/dt, t_i = t + c_i dt, and
8299 //
8300 //   F_j = lapScale K Y_j + bc(t_j) + s(Y_j).
8301 //
8302 // Note bc enters with coefficient 1 while the B and F terms carry 1/g:
8303 // that falls out of dividing the stage equation by g dt. Getting it
8304 // wrong would be invisible on this MMS suite, where every case is
8305 // zeroGradient and bc is identically zero - hence this note.
8306 //
8307 // The shape is the same one-step equation the Picard branch below
8308 // solves, which is why one matrix, one cached factorisation and one
8309 // fixed-point iteration serve every stage.
8310 //
8311 // Stiffly accurate (c_s = 1, b = last row): V^{n+1} = Y_s, the last
8312 // stage IS the answer and there is no separate combination step.
8313 //
8314 // Only Picard is wired up. JFNK and diagonalIion would each need their
8315 // per-stage Jacobian machinery threaded through, which is separate
8316 // work; an unknown scheme name is a FatalError upstream, so nothing
8317 // reaches here by accident.
8318 // ESDIRK runs only while it is supplying the first bdfOrder-1 starting
8319 // values. After that the history is full and BDF takes over for the
8320 // rest of the run - which, for anything but the opening steps, is the
8321 // whole run.
8322 const bool esdirkThisStep =
8323     (bdfOrder > 1 && esdirkStartup && nSteps < bdfOrder - 1);

```

Listing 21: Dónde el arranque ESDIRK es alcanzable, y sólo ahí.

10. El driver de Vm que manejan las EDO

Las EDO de estado se integran con sub-pasos *dentro* del paso de la PDE, y en cada sub-paso necesitan un V_m en un instante intermedio que el esquema no conoce. Cómo se rellena ese hueco **acota el orden de todo el solver**: una recta entre los dos extremos es $O(\Delta t^2)$, y ningún integrador temporal por encima de segundo orden puede mostrar el suyo detrás de eso.

No es una salvedad teórica, está medido. La clave `stateODEDriver` conmuta entre las dos opciones — `history`, el interpolante por los niveles que el esquema ya guarda, y `linear`, la rampa vieja — y `M21` corre BDF4 con cada una sin cambiar nada más: la rampa ajusta **2,04** contra **3,89** del interpolante, con el error del par más fino mayor por un factor 44,7. El 2,04 es exactamente lo que predice la cuenta: la rampa le erra por $O(\Delta t^2)$ punto a punto, que integrado sobre el paso es $O(\Delta t^3)$ local y $O(\Delta t^2)$ global.

`history` es el valor por defecto; `linear` existe para que ese costo siga siendo medible, porque un orden capado es todo lo que la corrida reporta — no hay error ni advertencia. El camino `gaussPointODE` usa siempre la rampa: subirlo obligaría a guardar cada nivel *en cada punto de cuadratura*.

`interpolateVmDriver` evalúa el polinomio de Lagrange a través de los voltajes que el esquema exterior ya calculó: dos nodos para las familias θ y BDF1, $k + 1$ niveles de historial para BDF- k , y los valores de etapa para el arranque ESDIRK. No cuesta ninguna evaluación extra — los nodos ya existen. El caso de dos nodos está escrito aparte y no pasa por el bucle, que es lo que garantiza que `backwardEuler` y `crankNicolson` sigan dando resultados idénticos bit a bit.

```

5284 inline scalar interpolateVmDriver
5285 (
5286     const VmDriverNodes& nd,
5287     const scalar s
5288 )
5289 {
5290     // Written out for two nodes rather than left to the general loop: this
5291     // is the path every non-ESDIRK scheme takes, and spelling it out is
5292     // what guarantees bit-identical results, not just equal ones.
5293     if (nd.nNodes <= 2)
5294     {
5295         return (1.0 - s)*nd.v[0] + s*nd.v[1];
5296     }
5297
5298     scalar result = 0.0;
5299
5300     for (label j = 0; j < nd.nNodes; ++j)
5301     {
5302         scalar Lj = 1.0;
5303
5304         for (label m = 0; m < nd.nNodes; ++m)
5305         {
5306             if (m != j)
5307             {
5308                 Lj *= (s - nd.s[m])/(nd.s[j] - nd.s[m]);
5309             }
5310         }
5311
5312         result += Lj*nd.v[j];
5313     }
5314
5315     return result;
5316 }

```

Listing 22: El interpolante, con el caso de dos nodos escrito aparte.

```

5318 void reactionRatesLinearVm
5319 (
5320     const VmDriverNodes& vmNodes,
5321     const scalar tau,
5322     const scalar dt,
5323     const scalar u1,
5324     const scalar u2,
5325     const scalar u3,
5326     scalar& du1dt,
5327     scalar& du2dt,
5328     scalar& du3dt
5329 )
5330 {
5331     const scalar alpha =
5332         dt > SMALL
5333         ? min(max(tau/dt, scalar(0.0)), scalar(1.0))
5334         : scalar(1.0);
5335 }

```

```

5336     const scalar VmTau = interpolateVmDriver(vmNodes, alpha);
5337
5338     reactionRates(VmTau, u1, u2, u3, du1dt, du2dt, du3dt);
5339 }

```

Listing 23: Dónde el driver entra a las EDO.

11. Reconstrucción a los puntos de integración

Con alto orden, V_m y los estados viven en los centros de celda pero la corriente iónica se evalúa en los puntos de Gauss. Estas rutinas hacen la reconstrucción LRE de ida y el promedio de vuelta.

`reconstructStatesAtIionIntegrationPoints` lleva un limitador de Barth–Jespersen, acotando la reconstrucción al rango de los vecinos. No es cosmético: sin él, el sobrepaso de alto orden sobre estados rígidos desborda.

```

4933 void reconstructVmAtIionIntegrationPoints
4934 (
4935     const volScalarField& Vm,
4936     const Switch useHighOrderVm,
4937     const highOrderInterp& LREInterp_Vm,
4938     const highOrderInterp& LREInterp_Iion,
4939     scalarField& VmIntegrationPoints,
4940     const Switch useOpenMP,
4941     const label openMPThreshold,
4942     const Switch profileTimings,
4943     FineGrainedTimings& fineTimings
4944 )
4945 {
4946     const fvMesh& mesh = Vm.mesh();
4947     const vectorField& C = mesh.C();
4948     const bool twoD = mesh.nGeometricD() == 2;
4949     const label nCells = mesh.nCells();
4950
4951     const CompactListList<point>& cellIionQuadP =
4952         LREInterp_Iion.cellQuadPoints();
4953     // Where each cell's quadrature points start in the flat array. Taken
4954     // from the container rather than counted along the loop: the running
4955     // counter this replaces was a dependency between iterations, and was
4956     // the only thing making either branch unparallelisable.
4957     const labelList& ipOffsets = cellIionQuadP.offsets();
4958
4959     const scalarField& VmI = Vm.primitiveField();
4960
4961     if (useHighOrderVm)
4962     {
4963         const label order = LREInterp_Vm.order();
4964
4965         const auto tDerivStart = std::chrono::steady_clock::now();
4966
4967         // Halo exchange: MPI, once, outside the threaded region.
4968         const auto& stencilData = LREInterp_Vm.mls().stencilData();
4969         const globalIndex& globalCells = stencilData.globalCells();
4970         const Map<FixedList<label, 2>>& remoteLoc =
4971             stencilData.remoteCellLocation();
4972         const CompactListList<label>& stencils = stencilData.cellsStencil();
4973
4974         const List<Field<scalar>> VmRemote =
4975             stencilData.remoteFieldPerProc(VmI);
4976
4977         // Coefficients of the local fit, already built and cached by the
4978         // library. The Hessian and third-derivative tables are empty below
4979         // the order that defines them, hence the guards.
4980         const CompactListList<vector>& gradCoeffs =
4981             LREInterp_Vm.QRGradCoeffs();
4982         const CompactListList<symmTensor>& hessCoeffs =
4983             LREInterp_Vm.cellHessianCoeffs();
4984         const CompactListList<highOrderInterp::symmTensor3Order>&
4985             thirdCoeffs = LREInterp_Vm.cellThirdDerivCoeffs();
4986

```

```

4987     const bool haveHess = (order >= 2);
4988     const bool haveThird = (order >= 3);
4989
4990     const auto tExpandStart = std::chrono::steady_clock::now();
4991
4992     // One pass: gather the stencil once, form every derivative, expand

```

Listing 24: Reconstrucción de Vm. Recortada. (*recortado; sigue en el fuente.*)

```

5105 void averageIntegrationPointFieldToCells
5106 (
5107     const scalarField& integrationPointValues,
5108     const highOrderInterp& LREInterp_Iion,
5109     volScalarField& field
5110 )
5111 {
5112     const fvMesh& mesh = field.mesh();
5113     const CompactListList<scalar>& cellIionQuadW =
5114         LREInterp_Iion.cellQuadWeightPhysical();
5115
5116     scalarField& cellValues = field.primitiveFieldRef();
5117     label integrationPointI = 0;
5118
5119     forAll(mesh.cells(), cellI)
5120     {
5121         scalar valueBar = 0.0;
5122         scalar wSum = 0.0;
5123
5124         forAll(cellIionQuadW[cellI], qI)
5125         {
5126             const scalar w = cellIionQuadW[cellI][qI];
5127             valueBar += w*integrationPointValues[integrationPointI];
5128             wSum += w;
5129             ++integrationPointI;
5130         }
5131
5132         cellValues[cellI] = valueBar/max(wSum, SMALL);
5133     }
5134
5135     field.correctBoundaryConditions();
5136 }

```

Listing 25: El promedio de vuelta a celdas.

12. Integración de las EDO, y el paralelismo por hilos

`advanceStateODE` avanza los estados en cada celda o punto de integración, con RK4 o RK45 adaptativo según el diccionario.

Acá están los **únicos** lazos con OpenMP de todo el archivo. Es importante para entender el rendimiento: los hilos reparten *estas* iteraciones y nada más — **el ensamblado no está paralelizado con hilos**, sólo por MPI. La región funciona porque cada punto integra su propia EDO sin hablar con ningún otro, y por eso el resultado es **bit-idéntico** con cualquier cantidad de hilos, cosa que **E20** mide en el otro solver dando exactamente cero.

El umbral por debajo del cual la región no se abre **está fijo en 256 en seis lazos**, ignorando la clave `stateODEOpenMPThreshold` del diccionario. Es un ítem abierto: la clave se lee y no hace nada.

```

5499 void advanceStateODE
5500 (
5501     const VmDriverNodes& vmNodes,
5502     const scalar u10ld,
5503     const scalar u20ld,
5504     const scalar u30ld,
5505     const scalar dt,
5506     const word& stateODESolver,
5507     const scalar stateODEInitialStep,
5508     const scalar stateODEAbsTol,
5509     const scalar stateODERelTol,

```

```

5510     const label stateODEMaxSteps,
5511     scalar& u1New,
5512     scalar& u2New,
5513     scalar& u3New
5514 )
5515 {
5516     if (dt <= SMALL)
5517     {
5518         u1New = u1Old;
5519         u2New = u2Old;
5520         u3New = u3Old;
5521         return;
5522     }
5523
5524     if (stateODESolver == "Euler")
5525     {
5526         scalar du1 = 0.0, du2 = 0.0, du3 = 0.0;
5527         reactionRatesLinearVm
5528         (
5529             vmNodes, 0.0, dt,
5530             u1Old, u2Old, u3Old,
5531             du1, du2, du3
5532         );
5533
5534         u1New = u1Old + dt*du1;
5535         u2New = u2Old + dt*du2;
5536         u3New = u3Old + dt*du3;
5537         return;
5538     }
5539
5540     if (stateODESolver == "RK4")
5541     {
5542         rk4StateStep
5543         (
5544             vmNodes, 0.0, dt, dt,
5545             u1Old, u2Old, u3Old,
5546             u1New, u2New, u3New
5547         );
5548         return;
5549     }
5550
5551     // Validated at start-up in createFields.H, so this is a guard, not a
5552     // reachable branch.
5553     if (stateODESolver != "RK45")
5554     {
5555         FatalErrorInFunction
5556         << "Unknown state ODE solver: " << stateODESolver << nl
5557         << "Valid options are RK45, RK4 and Euler"
5558         << exit(FatalError);

```

Listing 26: El avance de los estados. Recortado. *(recortado; sigue en el fuente.)*

```

5402 void rkf45StateStep
5403 (
5404     const VmDriverNodes& vmNodes,
5405     const scalar tau,
5406     const scalar h,
5407     const scalar dt,
5408     const scalar u1,
5409     const scalar u2,
5410     const scalar u3,
5411     scalar& u1Fifth,
5412     scalar& u2Fifth,
5413     scalar& u3Fifth,
5414     const scalar stateODEAbsTol,
5415     const scalar stateODERelTol,
5416     scalar& err
5417 )
5418 {
5419     scalar k11 = 0.0, k12 = 0.0, k13 = 0.0;
5420     scalar k21 = 0.0, k22 = 0.0, k23 = 0.0;
5421     scalar k31 = 0.0, k32 = 0.0, k33 = 0.0;
5422     scalar k41 = 0.0, k42 = 0.0, k43 = 0.0;
5423     scalar k51 = 0.0, k52 = 0.0, k53 = 0.0;
5424     scalar k61 = 0.0, k62 = 0.0, k63 = 0.0;

```

```

5425     reactionRatesLinearVm
5426     (
5427         vmNodes, tau, dt,
5428         u1, u2, u3,
5429         k11, k12, k13
5430     );
5431     reactionRatesLinearVm
5432     (
5433         vmNodes, tau + h/5.0, dt,
5434         u1 + h*(1.0/5.0)*k11,
5435         u2 + h*(1.0/5.0)*k12,
5436         u3 + h*(1.0/5.0)*k13,
5437         k21, k22, k23
5438     );
5439     reactionRatesLinearVm
5440     (
5441         vmNodes, tau + 3.0*h/10.0, dt,
5442         u1 + h*((3.0/40.0)*k11 + (9.0/40.0)*k21),
5443         u2 + h*((3.0/40.0)*k12 + (9.0/40.0)*k22),
5444         u3 + h*((3.0/40.0)*k13 + (9.0/40.0)*k23),
5445         k31, k32, k33
5446     );
5447     reactionRatesLinearVm
5448     (
5449         vmNodes, tau + 3.0*h/5.0, dt,
5450         u1 + h*((3.0/10.0)*k11 - (9.0/10.0)*k21 + (6.0/5.0)*k31),
5451         u2 + h*((3.0/10.0)*k12 - (9.0/10.0)*k22 + (6.0/5.0)*k32),
5452         u3 + h*((3.0/10.0)*k13 - (9.0/10.0)*k23 + (6.0/5.0)*k33),
5453         k41, k42, k43
5454     );
5455     reactionRatesLinearVm
5456     (
5457         vmNodes, tau + h, dt,
5458         u1 + h*((-11.0/54.0)*k11 + (5.0/2.0)*k21 - (70.0/27.0)*k31 + (35.0/27.0)*k41),
5459         u2 + h*((-11.0/54.0)*k12 + (5.0/2.0)*k22 - (70.0/27.0)*k32 + (35.0/27.0)*k42),
5460         u3 + h*((-11.0/54.0)*k13 + (5.0/2.0)*k23 - (70.0/27.0)*k33 + (35.0/27.0)*k43),
5461

```

Listing 27: El paso adaptativo. Recortado. (*recortado; sigue en el fuente.*)

13. main(): montaje

`main()` arranca leyendo la malla y los diccionarios, construye los operadores LRE, y ensambla M y K **una sola vez** — son constantes, porque la no linealidad está en el término iónico y no en la difusión. Por eso en el desglose de tiempos el `setup` aparece separado del `loop`.

Es también donde se fija `gRowStart`, y donde viven los dos volcados de diagnóstico que la verificación usa, los dos detrás de una variable de entorno y gratis cuando están apagados: `CF_DUMP_K` escribe K como tripletes (fila global, columna global, valor) y `CF_DUMP_STENCILS` escribe la membresía de los stencils. Son la base de M09, M10, M11 y M15.

```

6452 int main(int argc, char* argv[])
6453 {
6454     #ifdef __GLIBC__
6455         // Tighten the glibc allocator before any large allocation. The LRE
6456         // construction runs ~2.3M QR factorisations on a 3D unstructured tet
6457         // mesh with N=40 / p3; without these tunables glibc grows the heap by
6458         // several GB of unreclaimed fragments and the run OOMs on a 16 GB box.
6459         // - M_ARENA_MAX=2: prevents glibc from creating ~64 arenas (one set
6460         // per ODE OpenMP thread). LRE setup is serial; multiple arenas
6461         // just inflate the RSS.
6462         // - M_MMAP_THRESHOLD=64 KB: routes mid-sized allocations (Eigen
6463         // temporaries during QR) through mmap so they are returned to
6464         // the kernel individually when freed.
6465         // - M_TRIM_THRESHOLD=64 KB: lets glibc release sbrk-heap padding
6466         // back to the OS more aggressively.
6467         mallopt(M_ARENA_MAX, 2);
6468         mallopt(M_MMAP_THRESHOLD, 64*1024);
6469         mallopt(M_TRIM_THRESHOLD, 64*1024);

```

```

6470 | #endif
6471 |
6472 |     #include "setRootCaseLists.H"
6473 |     PetscSession petscSession(argc, argv);
6474 |     #include "createTime.H"
6475 |     #include "createMesh.H"

```

Listing 28: El arranque.

```

6761 |     gRowStart = LREInterp_Vm.globalCells().localStart();
6762 |     gRowStartSet = true;
6763 |
6764 |     Info<< "Distributed linear algebra: rows ["
6765 |         << gRowStart << ", " << gRowStart + mesh.nCells() << ") of "
6766 |         << LREInterp_Vm.globalCells().totalSize() << " global cells" << endl;
6767 |
6768 |     SpMat M;

```

Listing 29: Dónde se fija el offset global de fila.

```

2078 | void logMemoryCheckpoint(const char* stage)
2079 | {
2080 |     const scalar peakMemoryKB = peakResidentSetSizeKB();
2081 |     Info<< "Memory checkpoint [" << stage << "]: peakRSS_MB = "
2082 |         << peakMemoryKB/1024.0 << endl;
2083 | }

```

Listing 30: Los checkpoints de memoria que M17 usa para ajustar el techo.

14. main(): el lazo temporal y los métodos no lineales

Cada paso resuelve un sistema no lineal, y hay tres métodos: **Picard** con relajación, **JFNK** — Newton sin jacobiano explícito, con el producto matriz-vector aproximado por diferencias — y **diagonalIion**, que linealiza la corriente iónica por su derivada diagonal. **M08** verifica que los tres lleguen al mismo lugar, con una tolerancia *derivada* de la tolerancia no lineal en vez de elegida a mano.

Al agotar las iteraciones sin converger hay dos salidas posibles, y **los tres métodos las eligen ahora con la misma clave**: **false** descarta el paso y vuelve a los valores del inicio, **true** se queda con el iterado y lo dice. Antes Picard tenía su propia rama que aceptaba el último iterado *siempre*, así que el mismo solver tenía dos semánticas de error según el método. Se eliminó esa rama en vez de agregarle un rollback propio: las otras dos ya implementan las dos salidas, así que Picard cae solo en la correcta.

Medido forzando la no convergencia con **nonlinearIterations** 1 y tolerancias en $1e-30$, que es la única forma de hacerlo disparar — en los barridos de convergencia no ocurre nunca: con **false** el error final es **4.998e-04**, que es el de no haber avanzado el paso, y con **true** es **5.023e-06**. El rollback efectivamente revierte.

Un nombre de método inválido es **FatalError** y no un silencioso repliegue sobre Picard, que es lo que hacía el solver electro hasta que **E18** lo señaló.

La condición de parada del lazo tenía un error, y era invisible. El tiempo acumulado es una suma en punto flotante de Δt , y puede caer por debajo de **endTime** *por más que* **SMALL**: 160 pasos de $3,2 \times 10^{-3}$ suman 0,51199999999999868, a $1,3 \times 10^{-15}$ de 0,512. Con la condición vieja, **endTime** - **SMALL**, el lazo daba un paso de más y la corrida terminaba en 0,5152 sin avisar nada. Eso no es cosmético para un estudio temporal: los campos que se comparan entre corridas con distinto Δt dejan de ser del mismo instante. Ahora la condición es **endTime** - $\Delta t/2$, exacta porque acá Δt es constante. Lo encontró la guarda de un test de la suite, no una revisión del código, y el mismo error sigue presente en el solver electro.

```

6694 |     const bool usePicard = nonlinearMethod == "Picard";

```

```

6695 const bool useDiagonalIion = nonlinearMethod == "diagonalIion";
6696 const bool useJFNK = nonlinearMethod == "JFNK";
6697
6698 if (!usePicard && !useDiagonalIion && !useJFNK)
6699 {
6700     FatalErrorInFunction
6701         << "Unknown nonlinearMethod '" << nonlinearMethod << "'. "
6702         << "Valid options are Picard, JFNK and diagonalIion (exact"
6703         << " spelling, case-sensitive)."
6704         << abort(FatalError);
6705 }

```

Listing 31: El despacho de método no lineal, y el rechazo de un nombre inválido.

```

9464 // Modified for cardiacFoam: Picard used to have its own branch here
9465 // that accepted the last iterate unconditionally, so the same solver
9466 // had two error semantics depending on the method. All three now obey
9467 // nonlinearAcceptUnconverged: false rolls the step back, true keeps the
9468 // iterate and says so.
9469 if (!nonlinearConverged && !nonlinearAcceptUnconverged)
9470 {
9471     WarningInFunction
9472         << "Nonlinear solver did not converge at t = " << t
9473         << " after " << nonlinearIters << " iterations."
9474         << " Rolling back Vm, Iion and states to the values"
9475         << " at the beginning of the time step." << nl
9476         << "Set 'nonlinearAcceptUnconverged true;' in"
9477         << " spatialIntegrationProperties to accept unconverged"
9478         << " iterates instead." << endl;
9479
9480     VmGuess.primitiveFieldRef() = VmOld.primitiveField();
9481     applyExactVmBoundaryValues(VmGuess, t, dim);
9482     u1Guess.primitiveFieldRef() = u1Old.primitiveField();
9483     u2Guess.primitiveFieldRef() = u2Old.primitiveField();
9484     u3Guess.primitiveFieldRef() = u3Old.primitiveField();
9485     updateStateBoundaryValues(u1Guess, u2Guess, u3Guess, t, dim);
9486     u1Guess.correctBoundaryConditions();
9487     u2Guess.correctBoundaryConditions();
9488     u3Guess.correctBoundaryConditions();
9489     IionGuess.primitiveFieldRef() = Iion.primitiveField();
9490     IionGuess.correctBoundaryConditions();
9491     nonlinearRolledBack = true;

```

Listing 32: Las dos salidas al agotar las iteraciones, ahora comunes a los tres métodos.

```

7325 // Modified for cardiacFoam: stop within half a step of endTime, not within
7326 // SMALL. The accumulated time is a floating-point sum of dt, and it can land
7327 // just BELOW endTime by more than SMALL (1e-15): 160 steps of 3.2e-3 sum to
7328 // 0.51199999999999868, which is 1.3e-15 short of 0.512. The old test then
7329 // took one extra step and the run finished at 0.5152 with no warning - so a
7330 // field or an error "at endTime" belonged to a different instant, and a
7331 // temporal-order study comparing runs at different dt compared fields at
7332 // different times. Half a step is exact because dt is constant here, and it
7333 // leaves every run that was already correct with the same step count:
7334 // checked across 159 151 existing summaries, none of which had an extra step.
7335 while (runTime.value() < runTime.endTime().value() - 0.5*dt)
7336 {

```

Listing 33: El lazo que ya no da un paso de más, y por qué.

15. Índice de funciones

Todas las funciones del espacio de nombres anónimo, en orden de aparición. Las secciones anteriores recorren las principales; ésta es el mapa completo para ubicarse en el archivo.

línea	función	líneas
94	globalRowStart	15
117	checkPetscError	10

línea	función	líneas
139	buildDistributedMat	85
257	clear	15
273	reset	19
343	requireOneOf	31
385	usesPetscBackend	5
409	resolvePetscKspPc	18
438	petscKspTypeForLinearSolver	12
460	petscParallelPcTypeName	13
476	isGmresType	4
532	copyEigVecToPetscVec	12
546	copyPetscVecToEigVec	16
566	setPetscOptionsPrefix	11
802	updateValues	65
868	solve	54
935	petscShellMatMult	17
962	petscShellPCApply	17
1050	initialise	153
1403	computeF	25
1418	if	4
1431	computeG	21
1456	exactVm	4
1463	exactU1	4
1469	exactU2	4
1476	exactU3	4
1483	exactFieldValue	20
1507	reactionRates	18
1530	ionicCurrentPDE	15
1550	vmSourcePDE	13
1568	computeBeta	24
1595	estimatedN	8
1608	characteristicDx	35
1648	volumeWeightedL1	7
1658	volumeWeightedL2	7
1667	linfNorm	5
1676	cellErrorNorm	4
1765	esdirk3Tableau	37
1805	timeSchemeCoeffs	71
1879	normalizedMassMatrixType	19
1899	computeFieldErrorSummary	9
1911	quadraticForm	14
1933	reconstructFromTaylor	24
1961	fieldToEigVec	12
1976	sourceToEigVec	4
1984	eigVecToField	9
1996	relativeL2Difference	29
2033	gSquaredNorm	6
2042	gNorm	4
2049	gDot	6
2058	relativeL2Norm	4
2064	peakResidentSetSizeKB	10
2078	logMemoryCheckpoint	6

línea	función	líneas
2086	addDiagonalToMatrix	15
2103	solveGMRES	95
2200	solveLeftPreconditionedGMRES	97
2301	addTripletIfNeeded	13
2319	assembleDiagonalMassMatrix	20
2347	assembleConsistentMassMatrixHO	222
2574	orthogonalDiffusionCoeff	14
2591	normalDiffusivity	4
2600	stabilisationFaceCoeff	17
2621	addCellGradientDotCoeffs	46
2677	cellTaylorEntryCoeff	31
2717	buildCellTaylorExtrapolationRow	32
2752	addRemoteRowCoeffs	19
2778	buildCellGradientDotRow	27
2822	exchangeCoupledFaceRows	78
2906	addCellTaylorExtrapolationCoeffs	46
2963	addRhieChowInternalJumpCoeffs	40
3011	addRhieChowCoupledJumpCoeffs	40
3055	addRhieChowBoundaryJumpCoeffs	22
3083	assembleStandardOrthogonalStiffnessMatrix	295
3392	assembleHighOrderStiffnessMatrix	506
3903	assembleStandardOrthogonalBoundaryVector	69
3976	assembleHighOrderBoundaryVector	90
4070	solveSparseSystemEigen	72
4149	solveSparseSystem	74
4227	appendTetQuadraturePointsAndWeights	58
4290	cellQuadraturePointsAndWeights	71
4365	updateStateBoundaryValues	36
4405	applyExactVmBoundaryValues	32
4440	fillExactFields	60
4506	computeHighOrderLaplacian	89
4598	computeCellCentredIion	34
4634	computeCellCentredVmSource	34
4673	stencilValue	17
4715	reconstructStatesAtIionIntegrationPoints	189
4933	reconstructVmAtIionIntegrationPoints	168
5105	averageIntegrationPointFieldToCells	32
5138	clampPhysical	13
5155	initialiseIionIntegrationPointStates	26
5185	computeIionFromIntegrationPoints	32
5219	computeVmSourceFromIntegrationPoints	32
5284	interpolateVmDriver	33
5318	reactionRatesLinearVm	22
5343	rk4StateStep	54
5402	rkf45StateStep	92
5499	advanceStateODE	125
5627	updateIntegrationPointStatesODE	56
5711	updateStateFieldsODE	72
5784	computeGaussReconstructedError	109
5894	cellAsReconstructionError	14
5909	computeIntegrationPointStateError	65

línea	función	líneas
6452	<code>main</code>	3479